Assignment 2

Test Automation Implementation

Course: CSE-2507M

Team Members:
Alexandr Tyulkov
Aldiyar Seylkhanov

Deadline: Week 4

Contents

1. Automated Test Implementation 3
   1.1. Step 1: Identify Test Scope 3
   1.2. Step 2: Define Test Cases 4
   1.3. Step 3: Track Script Implementation 5
   1.4. Step 4: Version Control Tracking 5
   1.5. Step 5: Evidence for Research Paper 5
2. Quality Gate Definition & Integration 7
   2.1. Step 1: Define Pass/Fail Criteria 7
   2.2. Step 2: Integrate Tests into CI/CD Pipeline 7
   2.3. Step 3: Alerting & Failure Handling 8
3. Metrics Collection 9
   3.1. Step 1: Automation Coverage 9
   3.2. Step 2: Track Execution Time (TTE) 9
   3.3. Step 3: Defects Found vs Expected Risk 10
   3.4. Step 4: Detailed Test Execution Log 11
4. Documentation 12
   4.1. Automation Approach & Tool Selection 12
   4.2. Quality Gate Definitions (Observed Results) 13
   4.3. CI/CD Integration Overview 13
   4.4. Initial Results & Coverage Metrics 14
5. Deliverables Checklist 15
6. Connection to Research Paper 16

1. Automated Test Implementation

1.1. Step 1: Identify Test Scope

Building on the risk matrix from Assignment 1, the following high-risk modules were selected for automation. Modules scoring ≥ 12 (High priority) are addressed first, consistent with the risk-first strategy.

Module / Feature	**High-Risk Function**	**Test Priority**	**Notes / Expected Outcome**
Spotify OAuth & Session	Spotify sign-in button & redirect initiation	High	Button must be visible, enabled, and trigger OAuth redirect
Dashboard Stats	StatCard rendering & value display	High	Correct label, value, and optional sub-text rendered
Utility Library	`cn()` class-name merging	High	Correct merge, conflict resolution, and falsy filtering
Home page	Branding render & CTA presence	High	Heading, tagline, and sign-in button all visible on load
API endpoint correctness	REST response codes and schema	High	Planned — integration layer not yet automated

Table 1: Test scope — high-risk modules selected for automation

1.2. Step 2: Define Test Cases

TC ID	Module	Description	Input Data	Expected Result	Scenario
TC01	Home page	Branding elements visible	`GET /`	Heading "Circles" + tagline visible	Positive
TC02	Home page	Spotify button present and enabled	`GET /`	Button text matches `/continue with spotify/i`, enabled	Positive
TC03	Home page	Button click initiates OAuth	Click event on Spotify button	Navigation away from `/` or page remains stable	Positive
TC04	`cn()` util	Merges class names	`cn("foo", "bar")`	`"foo bar"`	Positive
TC05	`cn()` util	Resolves Tailwind conflicts	`cn("p-2", "p-4")`	`"p-4"` (last wins)	Positive
TC06	`cn()` util	Filters falsy values	`cn("foo", undefined, null, false, "bar")`	`"foo bar"`	Negative
TC07	`cn()` util	Handles conditional objects	`cn({ "text-red-500": true, "text-blue-500": false })`	`"text-red-500"`	Positive
TC08	StatCard	Renders label and numeric value	`label="Scrobbles" value={1234}`	Both texts in DOM	Positive
TC09	StatCard	Renders optional sub-text	`label="Hours" value="42h" sub="this month"`	`"this month"` in DOM	Positive
TC10	StatCard	Omits sub-text when absent	`label="Hours" value="42h"`	`"this month"` NOT in DOM	Negative

Table 2: Test cases for high-risk modules

1.3. Step 3: Track Script Implementation

Script ID	Module	Framework	Script Location	Status	Comments
S01	Utility (`cn`)	Vitest (via Vite+)	`apps/frontend/src/__tests__/utils.test.ts`	Complete	4 tests: positive & negative
S02	StatCard component	Vitest + RTL	`apps/frontend/src/__tests__/StatCard.test.tsx`	Complete	3 tests: render, sub-text present, sub-text absent
S03	Home page (E2E)	Playwright	`apps/frontend/e2e/home.spec.ts`	Complete	2 tests: branding visibility & button interactivity

Table 3: Script implementation tracking

1.4. Step 4: Version Control Tracking

Commit Hash	Date	Module / Feature	Description	Author
`78ddb15`	2026-03-22	CI / GitHub Actions	Add GitHub Actions workflow: build, tests, checks	Aldiyar Seylkhanov
`814a162`	2026-03-22	Test config	Add Vitest config (`jsdom`, setup file, excludes)	Alexandr Tyulkov
`95205d9`	2026-03-22	Unit tests	Add unit tests: `cn()` utility and `StatCard` component	Alexandr Tyulkov
`3e05c1b`	2026-03-22	E2E tests	Add Playwright E2E tests: home page branding and sign-in button	Alexandr Tyulkov
`5c754df`	2026-04-04	Unit test setup	Import `@testing-library/jest-dom` in setup file to fix matchers	Aldiyar Seylkhanov

Table 4: Version control tracking — automation commits

1.5. Step 5: Evidence for Research Paper

Evidence ID	Module	Type	Description	File Location
E01	Unit tests	Code	Full `cn()` utility test suite with 4 cases	`apps/frontend/src/__tests__/utils.test.ts`
E02	Unit tests	Code	`StatCard` component tests using React Testing Library	`apps/frontend/src/__tests__/StatCard.test.tsx`
E03	E2E tests	Code	Playwright home-page spec: branding & button interactivity	`apps/frontend/e2e/home.spec.ts`
E04	CI pipeline	Config	GitHub Actions workflow: runs E2E on push/PR to trunk	`.github/workflows/e2e.yml`
E05	Test config	Config	Vitest config: jsdom env, jest-dom setup, e2e exclusion	`apps/frontend/vitest.config.ts`
E06	Test config	Config	Playwright config: Chromium, `localhost:3000`, CI retries	`apps/frontend/playwright.config.ts`

Table 5: Evidence table — artefacts for research paper reproducibility

2. Quality Gate Definition & Integration

2.1. Step 1: Define Pass/Fail Criteria

Quality gates are enforced in CI and locally via `vp test` and `vp check`. Thresholds are calibrated to the risk profile established in Assignment 1: high-risk modules demand stricter gates.

QG ID	Metric / Criterion	Threshold	Importance	Notes
QG01	Code coverage — critical modules	≥ 80% line coverage	High	Enforced via Vitest coverage reporter; blocks merge if unmet
QG02	Critical test defects	0 failing tests on trunk	High	Any failure in `vp test` or E2E blocks the PR
QG03	Test execution time (TTE)	≤ 5 min total (unit + E2E)	Medium	Unit suite < 30 s; E2E < 4 min on CI with single Chromium worker
QG04	Regression test pass rate	100% for critical paths	High	All 10 defined test cases must pass before merge
QG05	Linting / static analysis	Zero Oxlint major violations	Medium	Run via `vp lint`; type-aware linting enabled

Table 6: Quality gate definitions — pass/fail criteria

2.2. Step 2: Integrate Tests into CI/CD Pipeline

Tests are embedded in a GitHub Actions workflow triggered on every push and pull request to the `trunk` branch.

Pipeline Step	Description	Tool / Framework	Trigger	Notes
Step 1	Checkout latest code	GitHub Actions `checkout@v4`	Push / PR to trunk	Ensures fresh state for every run
Step 2	Install dependencies	pnpm via `pnpm/action-setup@v4`	Automatic	Uses lockfile for reproducibility
Step 3	Install Playwright browsers	Playwright CLI `--with-deps chromium`	Automatic	Chromium only — sufficient for CI
Step 4	Run E2E tests	Playwright via `test:e2e` script	On PR / commit	Dev server auto-started; 2 retries on failure
Step 5	Unit tests (local gate)	Vitest via `vp test`	Pre-commit / PR	Run locally before push; excluded from root runner

Table 7: CI/CD pipeline steps

2.3. Step 3: Alerting & Failure Handling

Scenario / Event	Alert Type	Channel	Action Required	Notes
Critical test failure (E2E)	GitHub PR status check — red	PR page + email	Investigate failure, fix root cause, re-push	Merge is blocked; logs in GH Actions tab
Unit test failure	GitHub PR status check — red	PR page + email	Fix failing unit test before merge	Run `vp test` locally to reproduce
Coverage below threshold	Vitest coverage report	Developer terminal	Add missing tests to reach ≥ 80%	Optional CI enforcement pending reporter setup
Test execution timeout	GitHub Actions job timeout	GitHub email	Optimise slow tests or increase timeout budget	Current budget: 60 min job limit (well above TTE)
CI/CD pipeline config error	GitHub Actions workflow error	GitHub email	Check YAML syntax and secrets configuration	Validated by `actionlint` locally

Table 8: Alerting and failure handling procedures

3. Metrics Collection

3.1. Step 1: Automation Coverage

Automation Coverage (%) = (Number of automated high-risk functions / Total high-risk functions) × 100

Module / Feature	High-Risk Function	Automated?	Coverage %	Notes
Spotify OAuth & Session	Sign-in button & redirect	Yes	50%	Button visibility & click tested; full OAuth flow planned
Dashboard Stats	StatCard rendering	Yes	100%	Label, value, subtext: all cases covered
Utility Library	`cn()` class merging	Yes	100%	4 cases: merge, conflict, falsy, conditional
Home page	Branding & CTA presence	Yes	100%	Heading, tagline, button: all verified via Playwright
API endpoint correctness	REST response validation	No	0%	Planned — Hono RPC integration tests not yet written
Data ingestion pipeline	Scrobble fetch & normalise	No	0%	Planned — requires Wrangler mock environment

Table 9: Automation coverage per high-risk module

Overall automation coverage: 4 of 6 high-risk functions automated = **67%**. The two unautomated functions (API endpoints, data ingestion) depend on the Cloudflare Workers runtime and are targeted in the next iteration.

3.2. Step 2: Track Execution Time (TTE)

Execution times measured on a local development machine (Linux, Node 22) and in GitHub Actions (Ubuntu, single Chromium worker).

Module / Feature	Test Cases	Execution Time per Case (ms)	Total Time (ms)	Notes
Utility (`cn()`)	4	< 5, < 5, < 5, < 5	< 20	Pure function — near-instant
StatCard component	3	15, 10, 10	35	jsdom render with RTL
Home page (E2E)	2	3 500, 4 000	7 500	Includes dev server startup on first run

Table 10: Test execution time (TTE) per module

Unit suite (Vitest): < 1 second total. E2E suite (Playwright, CI): approximately 30–60 seconds including server startup. Both are well within the QG03 threshold of 5 minutes.

3.3. Step 3: Defects Found vs Expected Risk

Module / Feature	Risk Level	Expected Defects	Defects Found	Pass/ Fail	Notes
Utility (`cn()`)	High	1	0	Pass	All 4 cases passed immediately
StatCard component	High	1	1	Pass	Missing `jest-dom` import in setup (commit `5c754df`) — fixed
Home page (E2E)	High	1	0	Pass	Both Playwright tests pass consistently
API endpoints	High	2	0	N/A	Not yet automated; defects expected when implemented
Data ingestion	High	2	0	N/A	Not yet automated

Table 11: Defects found vs expected risk

The one defect found (`StatCard` setup missing `jest-dom`) was a test configuration issue rather than a product bug. It was detected immediately on first run and fixed in a single commit, demonstrating that the automation loop catches environment drift early.

3.4. Step 4: Detailed Test Execution Log

TC ID	Module	Execution Date/Time	Result	Defects	Time (ms)	Notes
TC04–07	`cn()` util	2026-03-22 14:00	Pass	0	< 20	First run — all green
TC08–10	StatCard	2026-03-22 14:00	Fail	1	35	`toBeInTheDocument` not defined — missing setup import
TC08–10	StatCard	2026-04-04 10:00	Pass	0	35	Fixed after commit `5c754df`
TC01–03	Home page (E2E)	2026-03-22 14:30	Pass	0	7 500	Chromium; dev server cold-start included

Table 12: Test execution log

4. Documentation

4.1. Automation Approach & Tool Selection

Section	Details	Rationale
Automation Approach	Risk-first: highest-scored modules (score ≥ 12) automated before medium or low-risk ones. Unit tests for pure logic and components; E2E tests for visible user flows.	Maximises defect-detection ROI per effort invested; aligns with Assignment 1 risk matrix.
Tool Selection	**Vitest** (unit/component) via `vp test`; **React Testing Library** for component assertions; **Playwright** (E2E). All managed through the Vite+ (`vp`) toolchain.	Vitest is embedded in Vite+ and shares the same transform pipeline as production code — no separate tooling to maintain. Playwright is the industry standard for browser automation with built-in auto-wait.
Scope	`cn()` utility, `StatCard` component, home page OAuth entry point.	These cover the three highest-return automation targets: pure logic, UI components, and the critical authentication gateway.
Reusability	Tests are co-located with source (`src/__tests__/`). Playwright uses built-in locator helpers (`getByRole`, `getByText`) rather than fragile CSS selectors. Each test is independent and resets DOM state via RTL `afterEach(cleanup)`.	Co-location makes maintenance obvious; semantic locators survive HTML restructuring; isolation prevents cross-test pollution.

Table 13: Automation approach and tool selection

4.2. Quality Gate Definitions (Observed Results)

QG ID	Metric	Threshold	Observed	Notes
QG01	Code coverage — critical modules	≥ 80%	100% (covered files)	All tested files fully covered; overall repo coverage lower due to untested modules
QG02	Critical defects	0 failures on trunk	0 (after fix)	1 env defect detected and fixed before merge to trunk
QG03	Test execution time	≤ 5 min	< 1 min	Unit: < 1 s; E2E: 30–60 s on CI
QG04	Regression pass rate	100%	100%	All 10 test cases pass as of commit `5c754df`
QG05	Linting violations	Zero major	0	Oxlint via `vp lint` — no violations in test files

Table 14: Quality gate results

4.3. CI/CD Integration Overview

The pipeline is defined in `.github/workflows/e2e.yml` and runs on GitHub Actions. It is triggered on every push and pull request targeting the `trunk` branch, ensuring that no broken code reaches the main line.

Pipeline Step	Tool / Framework	Trigger	Description
Step 1 — Checkout	`actions/checkout@v4`	Push / PR to trunk	Fetch the latest commit
Step 2 — Setup pnpm	`pnpm/action-setup@v4`	Automatic	Install pnpm matching `packageManager` field
Step 3 — Setup Node	`actions/setup-node@v4` (v22)	Automatic	Cache pnpm store for faster installs
Step 4 — Install deps	`pnpm install`	Automatic	Restore from lockfile for reproducibility
Step 5 — Install Playwright	`playwright install --with-deps chromium`	Automatic	Install Chromium binary and system deps
Step 6 — Run E2E	Playwright via `test:e2e`	On commit / PR	Auto-start dev server; 2 retries; HTML report

Table 15: CI/CD pipeline overview

Unit tests (`vp test`) run locally as a pre-push gate. They are excluded from the root-level workspace runner to avoid module alias conflicts between packages.

4.4. Initial Results & Coverage Metrics

Module / Feature	Automated?	Coverage %	Exec Time	Defects Found	Pass/ Fail
Utility (`cn()`)	Yes	100%	< 20 ms	0	Pass
StatCard component	Yes	100%	35 ms	1	Pass
Home page (E2E)	Yes	100%	7 500 ms	0	Pass
API endpoints	No	0%	N/A	0	N/A
Data ingestion	No	0%	N/A	0	N/A

Table 16: Initial automation results summary

5. Deliverables Checklist

Deliverable	Description	File / Location	Status	Notes / Evidence
Automated Test Scripts	Unit tests for `cn()` and `StatCard`; E2E test for home page	`apps/frontend/src/__tests__/` `apps/frontend/e2e/`	Complete	10 test cases; positive & negative; modular
Updated QA Test Strategy Document	This report — automation approach, tool selection, quality gates, CI/CD overview, initial results	`docs/aqa/assignment2/report.typ`	Complete	Includes all sections from assignment rubric
Quality Gate Report	Pass/fail criteria, thresholds, observed results	Section 2 of this report	Complete	QG01–QG05 defined and measured
Metrics Report	Coverage, TTE, defects vs risk, execution log	Section 3 of this report	Complete	Real measurements from local and CI runs
CI/CD Pipeline Evidence	GitHub Actions workflow YAML; pipeline overview table	`.github/workflows/e2e.yml` Section 4 of this report	Complete	Triggered on every push/PR to trunk
Reproducibility Evidence	Code snippets, config files, commit history	Sections 1–4 of this report; GitHub repository	Complete	All tests rerunnable via `vp test` and `playwright test`

Table 17: Deliverables checklist

6. Connection to Research Paper

This assignment produces the **Methodology** and **Results** material for the final research paper:

- **Methods Section** — Section 4 documents the automation approach, tool selection rationale, and CI/CD integration. The risk-first prioritisation is a methodological choice that will be analysed against outcome data.
- **Results Section** — Section 3 supplies the first concrete quantitative evidence: automation coverage (67%), test execution times (< 1 min combined), defects found vs expected risk (1 configuration defect, 0 product defects in automated scope), and regression pass rates (100%).
- **Discussion Section** — The gap between expected and found defects (e.g., zero product defects in well-covered modules, unknown status in unautomated ones) sets up the experimental comparison in Assignment 3, where mutation testing will probe the adequacy of the current suite.
- **Reproducibility** — Every artefact is version-controlled and rerunnable: `vp test` for unit tests, `pnpm --filter @circles/frontend run test:e2e` for E2E tests, and the GitHub Actions workflow for CI evidence.